

DualPath: 打破智能体 LLM 推理中的存储带宽瓶颈

Yongtong Wu^{1,3} Shaoyuan Chen^{2,3} Yinmin Zhong^{1,3} Rilin Huang¹

Yixuan Tan³ Wentao Zhang³ Liyue Zhang³ Shangyan Zhou³

Yuxuan Liu³ Shunfeng Zhou³ Mingxing Zhang² Xin Jin¹

Panpan Huang³

¹School of Computer Science, Peking University ²Tsinghua University ³DeepSeek-AI

摘要

多轮智能体 LLM 推理的性能正越来越多地由 KV-Cache 存储 I/O 而非计算主导。在当前广泛使用的解耦式架构中,从外部存储加载大规模 KV-Cache 会造成根本性不均衡: prefill engine 上的存储 NIC 被带宽打满,而 decode engine 上的存储 NIC 却处于空闲状态。这种不对称严重限制了系统整体吞吐量。

本文提出 DualPath, 这是一个通过双路径 KV-Cache 加载来打破该瓶颈的推理系统。除传统的 storage-to-prefill 路径之外, DualPath 引入了一条新的 storage-to-decode 路径: KV-Cache 先被加载到 decode engine, 再通过计算网络上的 RDMA 高效传输到 prefill engine。DualPath 将这一优化数据路径与全局 scheduler 结合起来; 该数据路径天然避免网络拥塞, 并避免干扰对延迟敏感的模型执行通信, 而 scheduler 则在 prefill 与 decode engine 之间动态均衡负载。

我们在三个模型和真实智能体工作负载上进行评估。结果表明, DualPath 在内部推理系统上最高可将离线推理吞吐量提升 1.87 $\times$; 在不违反 SLO 的前提下, 它还可将在线服务吞吐量平均提升 1.96 $\times$ 。

1 引言

大语言模型 (LLM) 正在快速从单轮聊天机器人 [16, 36] 和独立推理器 [36] 演进为智能体系统: 它们能够自主规划、调用工具, 并通过多轮交互解决真实世界任务 [8, 24, 31, 45, 48]。在这类场景中, LLM 不再服务于彼此隔离的 prompt; 相反, 它参与长时间运行的 session, 且上下文会随时间不断累积 [29]。随着智能体应用越来越普遍, 多轮 LLM 推理已经成为生产系统中的关键工作负载, 覆盖从代码助手 [47, 52] 到自主任务智能体 [28, 59] 等多类应用。

这种应用范式的变化也显著改变了 LLM 推理工作负载: 它从传统的人类-LLM 交互, 转向人类-LLM-环境交互, 即智能体范式。典型的人机交互模式是用户提供输入, 与 LLM 进行少数几轮交互, 然后消费 LLM 生成的结果。相比之下, 智能体 LLM 可能通过 Web 浏览器、Python 解释器等工具与外部环境交互, 持续数十轮甚至数百轮。尽管每一次工具调用或反馈本身都很短 (通常只有数百个 token), 上下文会跨轮次不断累积, 并可能增长到极长长度。因此, 智能体工作负载会变得高度 I/O-bound: 多轮、短追加的模式会带来极高的 KV-Cache 命中率, 通常 $\geq$

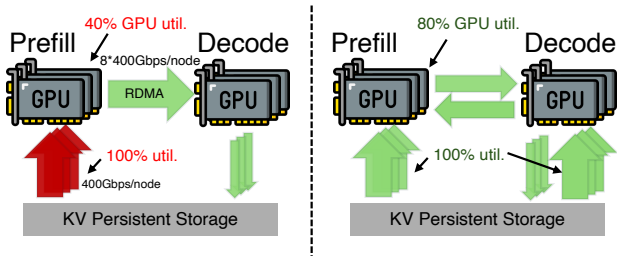


图 1: 现有瓶颈 (左) 与 DualPath (右)。

95% [7], 使得 KV-Cache 加载效率而非纯计算成为主导性能的因素。

为提升智能体工作负载下的吞吐量, 已有 LLM 推理系统逐渐收敛到一组常见架构模式: 逐层 *prefill* [17, 50]、*prefill-decode* (PD) 解耦 [37, 55, 58], 以及外部 KV-Cache 存储 [18, 30, 38]。在这些系统中, *prefill* engine 以逐层方式加载 KV-Cache, 以便在单个 batch 中容纳尽可能多的请求。当 *prefill* 完成后, *decode* engine 通常通过高性能 RDMA 网络从 *prefill* engine 接收 KV-Cache。随后 *decode* engine 生成 token, 并将其 KV-Cache 存入分布式存储, 以支持跨轮次复用。

然而, 该架构也引入了一个关键限制。如 Figure 1 所示, *prefill* engine 必须从远端存储加载大量 KV-Cache。因此, *prefill* 侧存储网络带宽会成为整个系统的吞吐量瓶颈, 即使 *decode* engine 通常拥有大量未被使用的存储网络带宽。

这种不均衡暴露了现有设计中的根本低效: 不同 engine 之间的存储网络带宽利用不均。*prefill* engine 的带宽长期饱和, 而 *decode* engine 的带宽利用不足。简单为 *prefill* engine 增配带宽代价高昂, 在通用集群中也往往不现实。因此, 更有前景的做法是利用并汇聚所有 engine 的可用 I/O 带宽, 而不是只让 *prefill* engine 承担全部压力, 从而加速智能体 LLM 工作负载中的 KV-Cache 加载。

已有研究尝试缓解 KV-Cache 加载瓶颈。Mooncake [38] 将 KV-Cache 缓存在分布式 DRAM 池中, 并采用亲和性感知 scheduler 来最大化 DRAM KV-Cache 命中率。然而, 它无法用于内存受限场景, 例如 RL 的 rollout 阶段, 因为此时 DRAM 会被用于保存从 HBM 卸载下来的大量训练状态。在工作集极大的

场景 (如在线服务) 中, 考虑 DRAM 与 SSD 的成本差异, 该方案也不具备成本效率。其他工作减少需要取回的 KV-Cache 数据量 [19], 或降低取回开销 [22, 51]。然而, 它们并未解决不同 engine 之间存储 I/O 不均衡所造成的内在低效。

本文提出 DualPath, 一个新的 LLM 推理系统, 用于重新思考现代智能体推理架构中的 KV-Cache 加载方式。DualPath 的核心洞察是: KV-Cache 加载并不必须以 *prefill* 为中心。现有系统总是直接从存储将 KV-Cache 加载到 *prefill* engine, 因而无法利用 *decode* engine 的远端存储带宽。DualPath 利用这一观察, 启用**双路径 KV-Cache 加载**: 除传统 *storage-to-prefill* 路径之外, KV-Cache 也可以先加载到 *decode* engine, 再通过高性能 RDMA 传输到 *prefill* engine。通过在两条路径之间动态选择, DualPath 重新分配网络负载并缓解 *prefill* 侧带宽压力。

实现这一设计带来两个挑战。第一, 引入额外加载路径会产生复杂流量模式, 并可能干扰模型执行中的 *collective* 原语; 若管理不当, 会降低整体性能。第二, 系统必须在动态且异构的工作负载下在线决定使用哪条加载路径, 并同时保证 GPU 与 NIC 的负载均衡。为解决这些挑战, DualPath 采用了: (1) 优化的双路径加载数据通路设计, 在常见 P/D 比例下不引入固有拥塞; (2) 以 NIC 为中心的流量管理方式, 将 KV-Cache 流量与对延迟敏感的模式推理通信隔离; (3) 动态调度策略, 在 *prefill* 与 *decode* engine 之间联合均衡计算和网络利用率。

我们在现代推理栈之上实现 DualPath, 并使用具有长上下文和高缓存复用率的代表性智能体工作负载进行评估。实验表明, DualPath 在保持 token 间延迟的同时, 显著提升系统吞吐量和首 token 延迟表现。在智能体推理场景中, DualPath 对离线推理最高可将端到端吞吐量提升 1.87 $\times$, 并使在线服务吞吐量平均提升 1.96 $\times$ 。

总之, 本文做出三项贡献:

- 我们识别出多轮智能体 LLM 工作负载的 I/O-bound 特性, 并说明在现代 LLM 推理架构下, KV-Cache 加载会主导系统性能。

- 我们提出 DualPath，一个引入双路径 KV-Cache 加载的推理系统，利用 decode engine 带宽解决 prefill 侧瓶颈。
- 我们设计并评估了一个工作负载感知调度算法，该算法动态均衡计算和网络资源，在真实工作负载上显著改善负载均衡。

2 背景

2.1 LLM 推理基础

LLM 推理正在成为近年来最重要的系统工作负载之一。主流 LLM 采用仅解码器 Transformer 架构，由堆叠的 attention 层和前馈网络 (FFN) 组成。attention 层支持请求内部 token 之间的交互，而 FFN 则独立处理各个 token。模型基于已有 token 预测后续 token，并将 attention 的 key 和 value 作为 KV-Cache 存储在 HBM 中，以避免重复计算。

PD 解耦推理。 *Prefill-decode (PD)* 解耦 [37, 58] 将 prefill 阶段与 decode 阶段分离，并分别分配给专用的 prefill engine (PE) 和 decode engine (DE)。这两个阶段呈现出不同的计算和内存访问模式：prefill 计算密集且通常以 batch 形式执行，而 decode 受内存限制且对延迟敏感。在 PD 解耦下，PE 加载命中的 KV-Cache 并执行 prefill；随后，它们将 KV-Cache 传输给 DE，由 DE 执行自回归解码。该设计减少了两个阶段之间的干扰，支持阶段特定优化，并提升可扩展性，因此已经成为现代 LLM serving 的事实标准架构。为支持多轮对话，KV-Cache 通常会存入分布式存储，以便跨轮次复用。

逐层 Prefill。 长上下文 prefill 会受到 HBM 容量瓶颈限制，因为整个 batch 的激活和 KV-Cache 都必须驻留在 HBM 中，这会迫使 batch size 受限，并导致 GPU 利用率较低。LayerKV [50] 和 PrefillOnly [17] 利用 prefill 计算中的强局部性解决这一问题：每一层只需要该层自己的 KV-Cache。因此，KV-Cache 可以按层分配和释放，GPU 在一个 forward batch 中只需保存一层的 KV-Cache。这会使有效 batch size (以

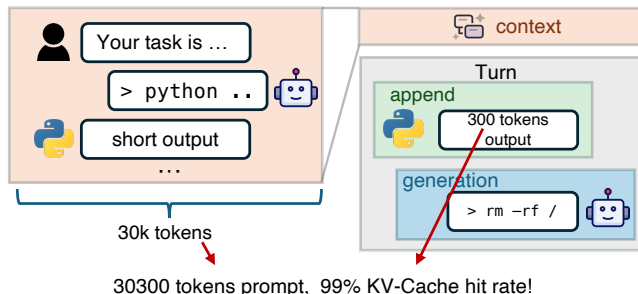


图 2: 智能体轨迹示例。

token 计) 近似提升为层数倍，从而提高 prefill 吞吐量。

2.2 LLM 的智能体式使用

LLM 越来越多地支撑智能体应用：这些应用在长期 session 中进行多轮推理，并与环境交互，例如执行终端命令、运行代码，或请求人类反馈。如 Figure 2 所示，在典型的一轮中，模型接收一个由先前上下文和新追加 token (通常是工具输出或用户输入) 组成的 prompt，并生成下一步动作或回复。一次完整智能体运行是一条包含数十轮甚至数百轮的轨迹：上下文逐轮增长，最长可达到一百万 token [4, 11]。由于大部分上下文都会跨轮复用，在我们的 trace 中通常超过 95% 的 token 会被复用，因此每轮中的绝大多数 token 都能命中 KV-Cache；只有新追加的上下文需要执行 prefill 计算。由于智能体轨迹极长，像 Mooncake [38] 这类基于 DRAM 和 HBM 的 KV-Cache 存储只能保存很小一部分 KV-Cache，因此必须使用容量更大且成本更低的外部 SSD 型 KV-Cache 存储 [13]。

智能体 LLM 推理工作负载也广泛存在于智能体 LLM 训练中，这类训练通常采用强化学习 (RL) 方法。在典型 RL 训练循环中，智能体 LLM 首先进入 rollout 阶段，被提示生成大量多步智能体轨迹。随后，这些轨迹会由单独的奖励模型打分。最后，LLM 参数被更新，以提高高分输出的概率并降低低分输出的概率。在 rollout 阶段，大量数据 (例如奖励模型和优化器状态) 会被卸载到主机 DRAM，进一步压缩可用于 KV-Cache 的 DRAM 空间。这进一步强化了对外部

高容量 KV-Cache 存储的需求, 以便高效容纳长智能体 rollout 上下文。

2.3 现代 AI 数据中心架构

现代 AI 数据中心是为大规模生成式 AI 训练和推理工作负载专门构建的逻辑超级计算机。例如, 在标准 NVIDIA DGX SuperPOD [32] 中, 每个节点配备 8 块 Hopper GPU, 并通过高速 NVLink 互联。每块 GPU 都配有一个专用 400 Gbps 计算 NIC (*CNIC*, 也称 east-west NIC), 用于最大化跨节点通信带宽。独立于计算网络之外, 每个节点还配有最高 400 Gbps 的存储 NIC (*SNIC*, 也称 south-north NIC), 用于快速访问数据集、模型 checkpoint 和磁盘上的 KV cache。

该架构的一个基本原则是计算网络与存储网络彼此隔离 [55]。这种隔离对于同时最大化存储性能和应用性能至关重要。通过将 GPU 之间高强度的 east-west 计算流量与存储流量隔离, 该架构可以防止二者互相干扰, 并显著降低计算通信延迟。即使在读取大型数据集或写入 TB 级模型 checkpoint 等数据密集型任务期间, 该设计也能保证 GPU 间通信保持高度可靠且可预测。

3 瓶颈与动机

我们观察到, 智能体推理任务中存在严重的 GPU 利用不足。进一步分析表明, 由于每个节点上单个存储 NIC 的带宽有限, KV-Cache 加载速度成为瓶颈。如下所述, 三个决定性因素共同导致了这一瓶颈。

第一, 智能体工作负载具有很高的 KV-Cache 命中率, 这意味着它们需要更多 *I/O*、更少计算, 从而形成严重 *I/O* 瓶颈。智能体工作负载天然具有长上下文、短追加和多轮交互特征。在每一轮中, GPU 需要从持久化存储读取整个上下文的 KV-Cache, 并对追加 token 执行 prefill 计算。我们从代表性代码任务中收集的 trace 显示, 平均轮数为 157, 说明 LLM 倾向于进行多轮交互。平均上下文长度为 32.7k, 而平均追加长度只有 429, 这对应 98.7% 的 KV-Cache 命中率。在这种场景下, cache-compute ratio, 即需要加载

表 1: 追加长度为 429 时, 不同上下文长度 (16k–64k) 下的 cache-compute ratio。除非特别说明, KV-Cache 数据类型默认使用 FP8。

Model	GB/PFLOP (16K–64K)
Qwen2.5-32B [42] (FP16)	117–267
GPT-OSS-120B [35]	47–95
Qwen3-235B-A22B [43]	39–60
DeepSeek-V3.2 660B [16]	13–36
DeepSeek-V3 660B [15]	4.8–5.8

的 KV-Cache 与所需计算量的比值, 对于 DeepSeek-V3.2 [16] 约为 22 GB/PFLOP, 对存储带宽构成显著瓶颈。需要注意的是, DeepSeek MLA 模型的 KV-Cache 大小已经高度优化; 对于 KV-Cache 更大的模型 (见 Table 1), 情况会更糟。DeepSeek-V3.2 的比值高于 DeepSeek-V3 [15], 这是因为它的稀疏 attention 设计降低了计算需求。

第二, 硬件演进趋势并不适合智能体推理工作负载。近年来, 网络带宽和 HBM 容量的增长落后于 GPU FLOPS 的增长, 这使智能体工作负载更容易撞上内存墙和通信墙。如 Figure 3 所示, 从 NVIDIA Ampere 到 Blackwell, *I/O*-compute ratio 下降了 14.4 $\times$ 。较低的 NIC 带宽限制了 KV-Cache 加载速度, 使 GPU 处于空闲状态。此外, 较小的 HBM 容量限制了 GPU kernel [10, 14, 27, 54] 可同时计算的 token batch size, 阻碍了 Tensor Core 等计算单元的充分利用 [17]。

第三, 现有 LLM 推理系统在不同 engine 类型之间存在严重的存储网络利用不均衡。在流行的 PD 解耦系统中, 命中 token 的 KV-Cache 只由 prefill engine 直接从远端存储加载。这种设计将全部存储 *I/O* 压力集中到 prefill 侧 SNIC 上, 而 decode engine 上的 SNIC 基本保持空闲。因此, 聚合存储网络带宽无法被充分利用。

上述分析表明, 在 PD 解耦架构上, 智能体推理的根本性能问题是 KV-Cache 取回带来的高 *I/O* 需求, 以及推理 engine 之间存储网络带宽利用不均。同

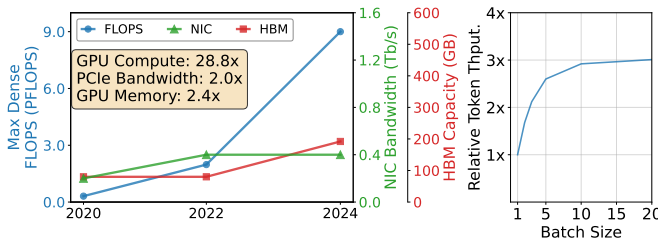


图 3: 左: NVIDIA GPU 的硬件趋势。右: 不同请求 batch size 下的相对 token 吞吐量 (每个请求包含 30K 上下文和 300 个追加 token)。

时, 我们观察到计算网络的流量具有间歇性模式: 模型推理中的 collective 操作以亚毫秒级 burst 形式出现, 而计算网络的聚合带宽又远高于存储网络。因此, 一个机会自然出现: 可以利用 decode 节点的 SNIC 带宽从存储加载 KV-Cache, 再利用更快计算网络的空闲带宽将其传回 prefill 节点。

4 DUALPATH 系统概览

为打破 prefill 侧存储 I/O 瓶颈, 我们提出一种**双路径加载架构**, 从根本上重新思考 PD 解耦推理中 KV-Cache 的取回方式。基于该架构, 我们设计并实现了 DualPath。DualPath 采用 §2 中介绍的两项被广泛采用的技术: (1) **PD 解耦** [37, 58], 将 prompt 处理与 decode 处理分离以提升效率。(2) **逐层 prefill**, 它避免了 LayerKV [50] 和 PrefillOnly [17] 所指出的 prefill engine 上 HBM 瓶颈, 并提高 GPU 利用率。

我们的系统由以下组件组成:

- **推理 Engine**。每个 engine 管理一块 GPU。Engine 分为用于 prefill 的 prefill engine (PE) 和用于 decode 的 decode engine (DE)。
- **流量管理器 (§5)**。每个 engine 都包含一个流量管理器, 用于执行: (1) Host-Device 内存拷贝 (H2D 与 D2H), (2) PE 与 DE 之间的 KV-Cache 传输, 以及 (3) 通过存储 NIC 从存储读取/向存储写入 KV-Cache。我们采用以 CNIC 为中心的流量管理方式, 详见 §5, 以防 KV-Cache 流量影响模型推理通信。

- **请求 Scheduler (§6)**。中央 scheduler 接收客户端请求并将其分发到各个 engine。它还负责在两条路径之间动态分配数据流量 (Figure 4)。

4.1 双路径加载

除传统的 *storage-to-prefill* 路径之外, DualPath 引入了一条新的 *storage-to-decode* 路径, 使 KV-Cache 可以先被加载到 decode engine, 再通过计算网络上的高带宽 RDMA 传输到 prefill engine。通过在两条路径之间动态分配负载, 系统能够聚合所有 engine 的存储 NIC 带宽, 包括原本空闲的 decode 侧 NIC, 并消除限制现有系统的不对称带宽饱和问题。这种方式将存储 I/O 从单一瓶颈资源转化为一个全局池化且可调度的容量。双路径的具体数据流如下。

为实现双路径加载, DualPath 在每个 PE 和 DE 上分配少量 DRAM 作为 buffer, 分别称为 *PE buffer* 和 *DE Buffer*。

Prefill PE 读取路径。首先, 命中 token 的 KV-Cache 从持久化存储读入 PE buffer (如 Figure 4a 中标签 1 和 2 所示)。在计算某个 attention 层之前, 该层的 KV-Cache 被传输到 PE HBM (3 和 4), 用于计算 cache-miss prompt token 的 KV-Cache。随后, 命中和未命中 token 的全部 KV-Cache 都被传输到 DE buffer, 形成完整的 prompt KV-Cache (5-7)。该过程 (3-7) 重复 n_{layer} 次。在 prefill forward 过程中, 传输会与计算重叠。

Prefill DE 读取路径。命中 token 的 KV-Cache 首先被读入 DE buffer (如 Figure 4b 中标签 1 和 2 所示)。在 PE prefill 期间, 对应层的 KV-Cache 从 DE buffer 读取, 同样与计算重叠 (3-5)。该过程重复 n_{layer} 次。当一层计算完成后, 只有 miss token 的 KV-Cache 会被传输到 DE buffer, 并与已有的 hit token KV-Cache 合并。

Decode 阶段。当 DE buffer 中收到完整的 prompt KV-Cache 后 (包括通过 PE 读取路径加载的 KV-Cache, 以及新追加 token 的 KV-Cache), decode

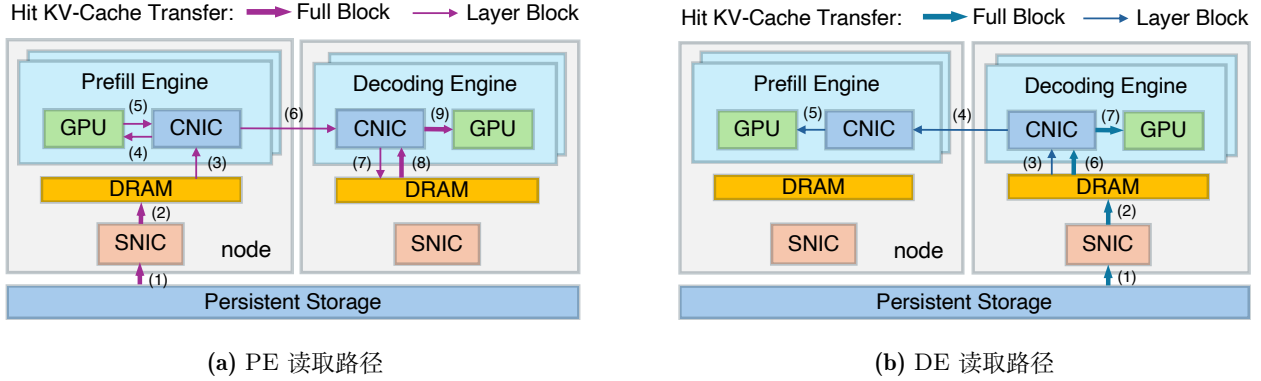


图 4: 双路径加载示意图。Scheduler 在两条路径之间动态分配数据流量。

阶段开始。DE 首先分配 HBM 并执行 host-to-device (H2D) 传输 (Figure 4a 中标签 8 和 9; Figure 4b 中标签 6 和 7), 随后在开始 decode 前释放 CPU 内存。DE buffer 的设计会对 DRAM 和 CNIC 施加带宽压力 (额外一次 H2D), 这本可通过 GPU Direct RDMA 直接绕过 buffer 来避免。然而, 由于该场景下 generation length 通常较短, time-to-first-token (TTFT) 在端到端请求时间中占据不可忽视的比例。引入 DE buffer 有助于降低 GPU 内存使用。在 decode 过程中, 一旦累积出一个完整 token block (例如 64 个 token), 系统会立即将其持久化到磁盘。

不同 Block 布局。我们采用两种不同 block 布局: *Full Block* 和 *Layer Block*, 分别包含所有层和单层。详细布局见 §A.5。对于所有与存储的交互, 我们使用 Full Block。在 PE 读取场景中, 将 KV-Cache 加载到 PE HBM 以及传输到 DE Buffer 都以逐层流式方式进行, 二者均使用 Layer Block。类似地, 对于 DE 读取路径, 从 DE Buffer 到 PE HBM 的传输也使用 Layer Block。

4.2 无瓶颈分析

我们证明, 在大多数合理 P/D 比例下, 系统可以充分打满所有存储 NIC, 而不会引入 compute-NIC 或 DRAM 瓶颈。我们假设 PCIe 拓扑配置合理 (每对 GPU-NIC 位于同一 PCIe switch 下)、任务调度负

载均衡、计算网络无拥塞, 并且存储读取带宽被充分利用。

符号。令 P 和 D 分别表示 prefill 节点和 decode 节点数量。每个节点有 g 块 GPU, 每块 GPU 配有一张带宽为 B 的 compute NIC。每台机器的存储带宽为 $s \times B$ (由该机器上的所有 engine 共享); M 为每台机器的内存带宽。

每个 PE-DE pair 的流量。我们假设存储读取带宽被充分利用, 且任务调度负载均衡。在负载均衡下, 存储 NIC 带宽被均匀共享。PE 读取路径 (Figure 4a 中所有步骤) 每个 pair 的流量为 $T_p = Bs/(Dg^2)$; DE 读取路径 (Figure 4b) 的流量为 $T_c = Bs/(Pg^2)$ 。链路流量等于使用该链路的所有 pair 流量之和。

PE CNIC 带宽分析。对于 PE CNIC, 存在 loopback 流量 (即不经过 switch 的 H2D 和 D2H), 因此无论读写操作如何, PCIe 侧总流量总是大于等于 switch 方向流量。因此, 我们只需要计算 PCIe 侧压力。读取操作包括 PE 路径 (3) 和 (5), 所有 pair 的总流量为:

$$2 \times T_p \times Dg = 2Bs/g \leq B \quad (1)$$

由于实践中总有 $s \leq g$, 读取方向始终无瓶颈。写入操作包括 PE 路径 (4) 和 DE 路径 (5), 总流量为:

$$(T_p + T_c) \times Dg = Bs/g \times (1 + D/P) \leq B \quad (2)$$

于是得到：

$$P/D \geq \frac{s}{g-s} \quad (3)$$

DE CNIC 带宽分析。对于 DE CNIC，读取操作包括 PE 路径 8 和 DE 路径 3/6，流量为：

$$(T_p + T_c \times 2) \times Pg = s/g \times (P/D + 2) \times B \leq B \quad (4)$$

于是得到：

$$P/D \leq \frac{g-2s}{s} \quad (5)$$

写入操作包括 PE 路径 7/9 和 DE 路径 7，流量为：

$$(2T_p + T_c) \times Pg \leq B \quad (6)$$

这意味着：

$$P/D \leq \frac{g-s}{2s} \quad (7)$$

DRAM 压力分析。DRAM 是半双工的，因此我们将读写压力相加。对于 PE MEM，压力为 $2sB$ ，通常不会超过内存带宽。对于 DE MEM，按照类似分析可得压力为 $(3 + 2P/D)Bs$ 。要求 DE MEM 压力不超过 M ，得到：

$$P/D \leq \frac{M/Bs - 3}{2} \quad (8)$$

总结。综合上述分析，可得：

$$\frac{s}{g-s} \leq P/D \leq \min \left\{ \frac{g-2s}{s}, \frac{g-s}{2s}, \frac{M/Bs-3}{2} \right\}. \quad (9)$$

对于 $(g=8, s=1)$ ，且 $M \approx 500 \text{ GB/s}$ 、 $Bs \approx 50 \text{ GB/s}$ 时，无瓶颈范围为 $\frac{1}{7} \leq P/D \leq \frac{7}{2}$ ，覆盖了大多数实际配置。

4.3 实践挑战

双路径架构从根本上重构了数据移动方式：KV-Cache 可以直接从存储加载到 prefill engine，也可以间接经由 decode engine 加载，从而聚合所有 engine 的存

储带宽并打破 prefill 侧 I/O 瓶颈。然而，要在实际系统中实现这一高层设计，会引入三个相互关联的挑战。我们先简要概述这些挑战，并在对应章节中展开。

细粒度数据传输 (§5)。逐层执行范式虽然对突破 HBM 容量限制至关重要，但会将 KV-Cache 切分为大量细粒度 block [37]。在存储、主机 DRAM 和 GPU HBM 之间传输大量细粒度数据块时，必须将额外开销降到最低，并与计算无缝重叠，才能真正获得吞吐量收益。

流量隔离 (§5)。DualPath 中复杂的数据路径会在计算网络和 PCIe 链路上引入额外 KV-Cache 传输流量。首要担忧是，这些流量可能干扰模型执行所必需的、对延迟敏感的 collective 通信，例如 expert parallel 中的 AllToAll [56]，以及 tensor/context parallel 中的 ReduceScatter/AllGather。由于这些 collective 通信对端到端推理延迟至关重要，关键挑战在于如何利用空闲 I/O 带宽，同时不降低模型推理性能。

动态负载均衡 (§6)。由于我们采用两条不同路径加载 KV-cache，系统必须及时决定每个请求使用哪条路径。朴素策略可能过载某一路径，从而重现原始瓶颈。流量 scheduler 必须实时平衡多个因素：存储 NIC 队列长度、GPU 计算负载，以及请求工作负载特征。

5 以 CNIC 为中心的流量管理器

现代 LLM 推理系统会采用一系列先进数据传输技术，例如片上 CUDA copy engine 和 GPUDirect Storage [34]，以便在存储、主机内存和 GPU HBM 之间高效移动数据。然而，所有这些机制都可能在模型执行期间干扰对延迟敏感的 collective 通信（例如 EP AllToAll）。这主要由两个原因造成：(1) 这些传输技术通常运行在独立路径上，无法共享与计算网络相同的 QoS 控制；(2) 现有 GPU 不支持 PCIe QoS [39]，因此难以保护模型推理通信免受其他竞争 PCIe 带宽的流量影响。此外，由于 collective 通信以快速的亚毫秒级 burst 形式出现，依赖软件流量整形器在这些高优先级流量窗口之间穿插低优先级 I/O 操作并不现实。

为解决这一问题,我们提出一种**以 CNIC 为中心的数据传输**方法,该方法已在我们的生产部署中广泛使用:所有进出 GPU 的数据流量,包括本地 H2D/D2H copy,都必须通过 GPU 配对的 CNIC,并使用 GPUDirect RDMA [33] 数据路径。通过将所有流量汇聚到计算网络上,我们可以利用计算网络原生 QoS 能力来执行严格的流量区分。

5.1 流量隔离

对于基于 InfiniBand 的网络,我们利用 virtual lane (VL) [5] 在不同流量类别之间实现隔离。所有模型推理通信流量被分配到专用高优先级 VL,而所有其他流量(包括 KV-Cache 传输)都被映射到单独的低优先级 VL。我们将所有网络 switch 和 NIC 的 VL arbiter 配置为加权轮询策略,为高优先级 VL 预留约 99% 的总带宽。剩余带宽分配给低优先级 VL,以避免饥饿。该配置保证模型执行流量几乎不受 KV-cache 传输影响,同时仍允许 KV-cache 流量机会性利用计算网络中的空闲带宽。详细配置见 §A.1。

尽管我们的实验在基于 InfiniBand 的网络上进行,同样的设计原则也自然适用于其他互连技术。DualPath 可以在 RDMA over Converged Ethernet (RoCE) 上运行,方法是结合 Traffic Class (TC) 和 Differentiated Services Code Point (DSCP) 标记 [6, 20],并配合硬件 packet queue。UnifiedBus [1] 和 Ultra Ethernet [9] 等新兴技术同样正在向面向异构流量的 QoS 机制收敛,可直接支持 DualPath 的需求。

5.2 CNIC 辅助的 KV-Cache 拷贝

现有 GPU 数据传输技术包括 GPUDirect Storage [34] 和 CUDA copy engine:前者从存储后端将 KV-Cache 加载到 GPU HBM,后者通过 PCIe 将 host DRAM 直接拷贝到 GPU。然而,这些方法无法将 KV-Cache 流量与模型执行中高优先级、对延迟敏感的 collective 通信隔离,会严重降低推理性能。

为解决现有方法的限制,我们采用 CNIC 辅助的 H2D/D2H 数据路径。对于 KV-Cache 加载,我们首先

从存储后端将 KV-Cache 读取到 host DRAM。随后,我们向 GPU 配对的 CNIC 提交 RDMA Write 请求,以执行本地 H2D copy。存储新生成的 KV-Cache 则采用对称流程:它先经由 CNIC 传输到 host DRAM,再通过存储网络持久化到存储后端。该设计将 CNIC 确立为所有 GPU PCIe 流量的中心 QoS scheduler,使其 VL arbiter 能够优先处理推理通信流量,并使用空闲 PCIe 带宽执行 KV-Cache 传输。

相比 GPUDirect Storage (直接将 KV-Cache 读入 GPU HBM) 和 CUDA copy engine (直接将 host memory 拷贝到 GPU HBM),该方法看起来绕了一段路;但据我们所知,它是目前唯一能实际保证 KV-Cache load/store 不会降低关键模型执行通信性能的方法。

我们还观察到,在处理大量小数据块时,CNIC 辅助 H2D 和 D2H 优于 CUDA copy engine。测量显示,通过 `cudaMemcpyAsync` 提交一次 copy 操作会带来约 5-7 μ s 的延迟开销。由于 CUDA driver 闭源,我们无法进一步拆解该开销。相比之下,提交一个 RDMA Write work request 只涉及用户态向 NIC register 的少量 mmio write,耗时约 1 μ s。此外,通过利用 *doorbell batching* [25], RDMA work submission 开销可以被显著摊薄。

6 自适应请求 SCHEDULER

尽管理论分析显示了较好的结果,但负载不均会降低硬件利用率;在该场景中,我们需要同时考虑两个维度的均衡:(1) NIC 流量,以及(2) GPU 利用率均衡。我们将调度划分为两个层次:inter-engine scheduling 负责将请求分配给一个 (PE, DE) pair,并为每个请求选择读取路径 (PE 或 DE);intra-engine scheduling 则决定每个 forward batch 中包含哪些请求用于计算。

6.1 Engine 间调度

我们将 engine 组织成 group,以降低 scheduler 压力。只有 engine rank 0 (称为 *Leader Engine*) 与 scheduler 交互。一个 group 中的所有 engine 要么全是 PE,要

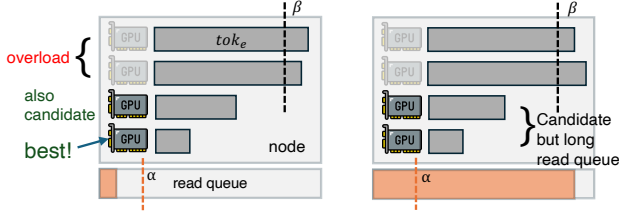


图 5: Inter-Engine PE Scheduling 示意图。八块 GPU 都位于同一个 PE engine group 中, scheduler 会选择最优 PE。

么全是 DE。同一节点上的所有 engine 保证属于同一 group。同一 group 中的所有 engine 会定期一起主动 fetch 任务。在 fetch 新请求时, 每个 engine e 上报: (1) seq_e , 即已分配给它但尚未完成的请求数量; (2) 这些 seq_e 个请求上的总 token 数 tok_e ; (3) engine e 所属节点 $n(e)$ 的磁盘读取队列长度 $read_q_{n(e)}$ 。GPU 负载、磁盘读取负载和网络负载都与 token 数强相关。因此, 我们使用 token 数作为代理指标, 并目标是在 engine 之间均衡该指标。

PE 调度。所有到达 scheduler 的请求进入等待队列, 并按 FIFO 顺序调度。当某个 PE group 发起 fetch request 时, 调度算法被调用。Inter-engine 调度过程示意图见 Figure 5。我们定义两个常数: 短读取队列阈值 α , 以及未完成 token 上限 β (以 token 为单位)。所有 engine 被划分为三类: (1) 过载 engine, 满足 $tok_e > \beta$; (2) 位于短磁盘读取队列节点上的 engine, 满足 $read_q_{n(e)} \leq \alpha$ 且 $tok_e \leq \beta$; (3) 位于较长磁盘读取队列节点上的 engine, 满足 $read_q_{n(e)} > \alpha$ 且 $tok_e \leq \beta$ 。我们不会向过载 engine 分配新请求。第二类 engine 优先级高于第三类, 因为它们所在节点的磁盘读取队列更短; 如果后续请求不足, 容易导致存储 NIC 利用不足。

若第二类非空, 我们将当前请求分配给第二类中 tok_e 最小的 PE; 否则若第三类非空, 则分配给第三类中 tok_e 最小的 PE。分配后, 我们更新被选中 PE 的 tok_e , 然后处理等待队列中的下一个请求。如果两类都为空, 则终止本次 fetch request, 并将已经分配的请求返回给 Leader Engine。

Algorithm 1: Inter-PE 调度算法

Data: 等待队列 Q , PE group G_{PE} , 每个 engine e 上报的负载指标

($read_q_{n(e)}, tok_e$), 其中 $n(e)$ 表示 engine e 所属节点, 常数 α 和 β

Result: 分配给 PE 的请求

每个 engine e 上报 ($read_q_{n(e)}, tok_e$);

将所有 PE 划分为三类: ;

$C_1 \leftarrow \{e \in G_{PE} : tok_e > \beta\};$

$C_2 \leftarrow \{e \in G_{PE} : read_q_{n(e)} \leq \alpha \wedge tok_e \leq \beta\};$

$C_3 \leftarrow \{e \in G_{PE} : read_q_{n(e)} > \alpha \wedge tok_e \leq \beta\};$

while Q is not empty **do**

$r \leftarrow Q$ 的队首请求;

if $C_2 \neq \emptyset$ **then**

$pe^* \leftarrow \arg \min_{e \in C_2} tok_e;$

else

if $C_3 \neq \emptyset$ **then**

$pe^* \leftarrow \arg \min_{e \in C_3} tok_e;$

else

终止本次 fetch request;

将已分配请求返回给 Leader

Engine;

break;

end

end

将请求 r 分配给 PE pe^* ;

更新 $tok_{pe^*} \leftarrow tok_{pe^*} + tokens(r);$

从 Q 中移除 r ;

end

DE 调度阶段 1: 跨 group。DE 调度分为两层, 且不保持全局 FIFO。系统有一个全局等待队列, 以及每个 DE engine group 的私有队列。新到达请求首先进入全局队列。当某个 DE group fetch 时, *group-level*

调度会从全局队列取出请求，并将每个请求分配给总 tok_e （该 group 内所有 engine 之和）最小的 group；这会在 group 之间均衡 token 数，从而均衡 NIC 和 GPU 负载。

DE 调度阶段 2: group 内。然后，我们计算该 group 中所有 DE 的剩余 HBM 总和，并从私有队列队首开始遍历，假设没有 HBM 碎片，计算有多少请求可以被调度。这些请求形成集合 R 。它是可调度请求集合的上界。随后，我们计算一个高 token 阈值 $Z = 1.05 \times (\sum_{r \in R} len_r + \sum_{e \in E} tok_e) / |E|$ 。

接下来，我们尝试弹出私有队列队首请求，并将其调度到某个 DE。在剩余 HBM 足够容纳该请求的 DE 中，我们划分两类：(1) 高 token DE，满足 $tok_e + len(r) > Z$ ；(2) 其余 DE。我们优先选择第 (2) 类，以保持 token 数均衡；第 (1) 类 DE 已经具有较高 GPU 和 NIC 压力。在第 (2) 类内部，我们选择 seq_e 最小的 DE 来均衡请求数量；如果第 (2) 类为空，则选择第 (1) 类中 tok_e 最小的 DE，以降低 HBM 耗尽和抢占风险。如果没有 DE 具备足够 HBM，则本次 fetch 结束，并返回已分配请求。

KV-Cache 读取任务调度。在为请求选定 PE 和 DE 后，我们选择读取队列更短的一侧执行读取。将请求拆成两部分并从两侧同时读取可能更好，我们将其留作未来工作。

6.2 Engine 内调度

只有 PE 需要 intra-engine scheduling，因为 DE 总是将所有请求放入 forward batch。Intra-engine 调度过程示意见 Figure 6。Data parallelism 被广泛用于 attention 层，尤其是 MLA 模型。在这种并行配置下，每块 GPU 服务不同请求集合。这可能导致所有 GPU 之间工作负载不均，而它们必须在 attention 阶段之后同步并一起进入 FFN 阶段，从而造成 GPU bubble 等待其他 peer。因此，我们需要确保它们具有相近的 attention 层执行时间，以最小化等待 bubble。

层时间估计。我们使用 FIFO packing 来决定一个 forward batch 中包含多少请求。Forward batch 中的每

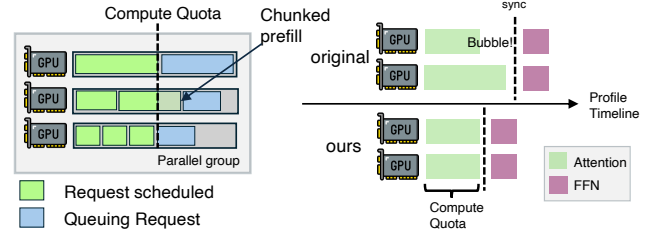


图 6: Intra-Engine 调度。左：基于 compute quota 的 batch 选择。右：应用 compute quota 前后的 GPU timeline。

个请求由一对 $(cached, bsz)$ 描述，其中 $cached$ 是 KV-Cache 已可用的 token 数（来自存储命中或先前 forward pass）， bsz 是本次 forward batch 中需要计算 KV-Cache 的 token 数。基于这些 pair，我们计算 attention 层的理论总计算量，并估计其执行时间。理论计算量与 wall-clock 时间之间的关系取决于硬件和并行配置，可以像已有工作 [17] 和 [2] 一样，通过预先 profiling 拟合得到。

算法。只要预测的 attention 层执行时间不超过预定义上限，我们就按 FIFO 顺序不断加入请求。该上限称为 *compute quota*。如果加入某个请求会超过该上限，我们会对 bsz 执行二分搜索，找到一个更小的 bsz' 来适配剩余 compute quota，并对该请求执行 chunked prefill。

7 评估

7.1 实现

我们基于内部推理框架实现 DualPath。对于 CUDA kernel，我们的内部框架采用 FlashMLA [27]、DeepGEMM [14] 和 DeepEP [56] 的组合，这与当前主流开源框架 [26, 57] 保持一致。DualPath 的实现大约在该框架之上修改了 5K 行代码。我们采用 3FS [13] 作为分布式存储，并使用类似 `io_uring` 的接口实现 kernel bypass。

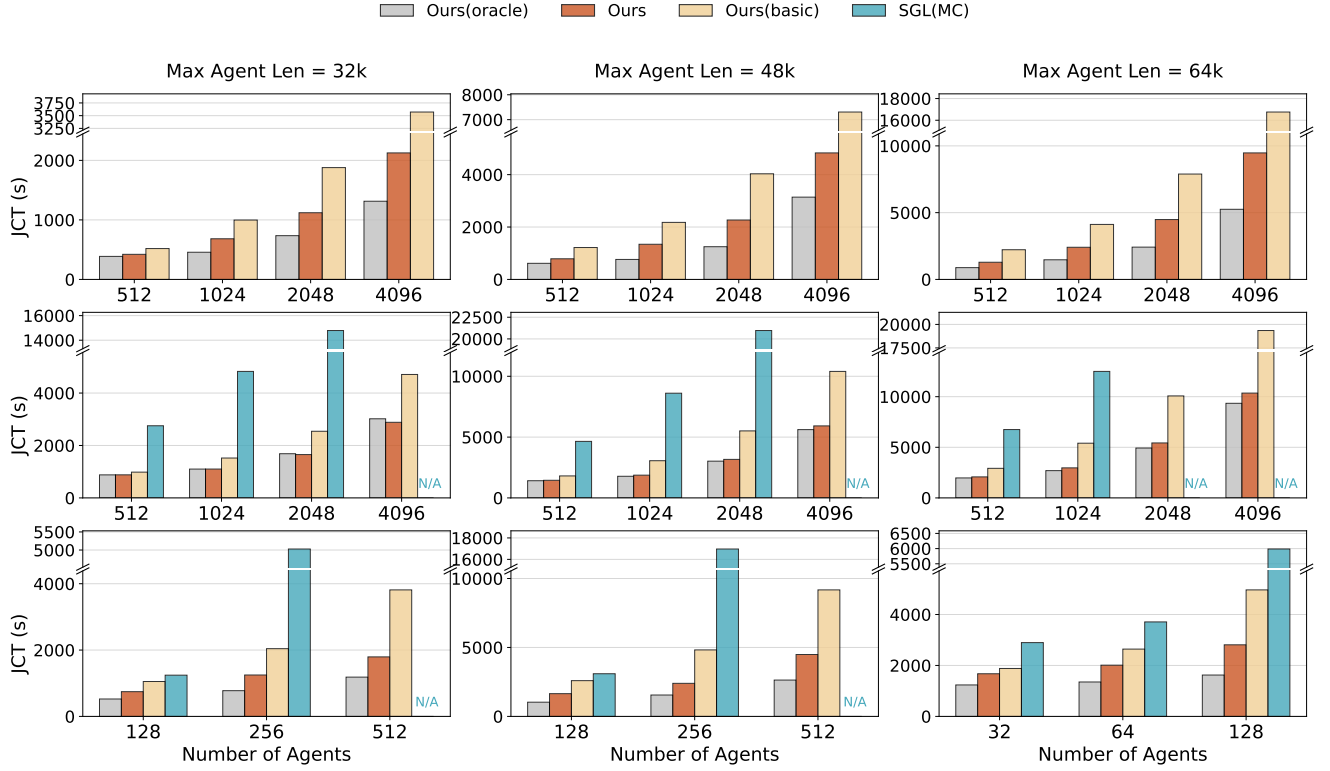


图 7: 不同智能体数量和最大智能体上下文长度下的离线推理性能。上: DS 27B。中: DS 660B。下: Qwen 32B。N/A 表示完成前遇到错误。

7.2 实验设置

测试平台。我们在一个使用 InfiniBand 互连的 GPU 服务器集群上进行实验。每台服务器配备 8 块 NVIDIA Hopper GPU 和双路处理器。此外，每个节点配置 8 张连接到 InfiniBand 网络的 400Gbps RDMA NIC，以及一张额外连接到 3FS 的存储 NIC。计算网络和存储网络在物理上隔离。我们的集群级 3FS 不包含内部 DRAM cache，并可打满存储 NIC 的 400Gbps 带宽。

模型。我们评估三个模型：(1) DeepSeek V3.2 [16] 660B，一个带 DeepSeek Sparse Attention 的 MoE 模型，记为 *DS 660B*；(2) DS 660B 的 27B 缩小版本，记为 *DS 27B*；(3) Qwen2.5-32B [42]，一个带 GQA 的 dense 模型，记为 *Qwen 32B*。DS 660B 和 Qwen 32B 对应 HuggingFace 上公开发布的 checkpoint。DS

表 2: 智能体 trace 数据集统计。

MaxLen	Turns	Append	Gen	Total	Context
32K	60	608	148	28639	17183
48K	106	474	172	42607	25120
64K	157	429	176	55958	32721

27B 是我们的内部实验模型，其架构与 DS 660B 类似。详细规格见 §A.2。

数据集。我们从生产智能体 RL 训练工作负载中收集了三个智能体 trace 数据集，最大上下文长度(MaxLen)不同。每个数据集包含 500 条轨迹。平均交互轮数 (Turns)、每轮平均追加和生成 token 数 (Append 和 Gen)、平均总 token 数 (Total) 以及平均上下文 token 数 (Context) 汇总于 Table 2。

基线。我们将 DualPath（记为 Ours）与以下基线比较：

- **SGL(MC)**: SGLang [57] (commit 19089aa)，启用 HiCache [40]、Mooncake [38] Store，并以 3FS 作为存储后端，同时使用 Mooncake Transfer Engine 支持 prefill-decode 解耦。由于 SGLang 不支持 DS 27B 这个缩小版本，我们没有在 DS 27B 上运行 SGL(MC)。
- **Basic**: 未修改的内部推理框架（详见 §7.1）。由于实现差异，直接比较 DualPath 和 SGL(MC) 并不公平。因此，我们只报告从 Basic 到 Ours 的性能提升。
- **Oracle**: 基于 DualPath，但绕过所有磁盘读取、D2H 与 H2D 传输，以及 inter-PD KV-Cache 传输。该配置代表假设 I/O 开销为零时的理论性能上界。

P/D 比例与并行方式。默认情况下，DS 660B 使用 2P4D，Qwen 32B 使用 1P2D，DS 27B 使用 1P1D（其中 1P1D 表示每一侧各用一个节点）。对于 DS 模型，我们使用 EP 和 DP。对于 Qwen 32B，DualPath 只使用 DP，而 SGL(MC) 使用 TP=8，因为 SGLang 中该模型不支持 DP attention。详细配置见 §A.4。

指标。对于离线推理场景，我们测量整个任务的 job completion time (JCT)。对于在线服务场景，我们测量 TTFT、TTST (Time to the second token) 和 TPOT。

7.3 离线批量推理

本节评估离线批量推理中的吞吐量性能，该场景对应 RL 训练中的 rollout 阶段。在该场景中， n 个智能体同时开始 rollout，我们测量所有请求完成时的 JCT。

改变智能体 Batch Size 与最大智能体长度 (MAL)。DualPath 在更大 batch size 和更长 MAL 下受益更多。Figure 7 报告了不同 batch size 和 MAL 下的 JCT。SGL(MC) 在我们的设置中遇到错误，无法完成部分大规模配置（标记为 N/A）。在 DS 660B 上，DualPath 相比 Basic 最高达到 1.87×，并展示出接近

Oracle 的性能，说明 KV-cache I/O 基本被消除。在 DS 27B 上，DualPath 相比 Basic 最高提升 1.78×，但由于 1P1D 下存储带宽有限，仍比 Oracle 慢 1.09–1.85× (Figure 8)。Qwen 32B 展现出与 DS 27B 类似的趋势。

改变追加长度与生成长度。当追加 token 和生成 token 较短时，DualPath 具有更大优势。更长的追加长度意味着更高 GPU 计算压力；更长的生成长度则会由于更大的 prefill 间隔时间而降低 KV-Cache 加载压力。为研究该因素影响，我们将每轮追加长度按常数因子缩放，然后在给定 MAL 处截断整条轨迹。生成长度采用同样处理。如 Figure 9 所示，随着追加长度增加，Basic 性能逐渐接近 DualPath 和 Oracle，而 DualPath 与 Oracle 性能变化很小，说明瓶颈持续位于 GPU 计算压力。相比 Basic，DualPath 在不同追加长度缩放下获得 1.82 – 1.99× 加速。生成长度缩放呈现类似趋势。

改变 Prefill-Decode 比例。在所有比例下，DualPath 相比 Basic 都展现出显著性能收益。我们在 DS 27B 上用 1P1D、2P1D 和 1P2D prefill-decode 比例进行 rollout 实验，以刻画 prefill 与 decode 阶段之间资源分配对系统整体性能的影响。如 Figure 8 所示，DualPath 在所有配置上平均加速 1.64×（最高 2.46×）。Basic 1P1D 与 Basic 1P2D 表现相近；DualPath 1P1D 与 Basic 2P1D 也相近，DualPath 2P1D 与 DualPath 1P2D 同样相近。这是因为每对系统拥有等价的可用存储带宽（Basic 只能使用 prefill 节点存储带宽，而 DualPath 可利用所有节点），这证明存储带宽是智能体场景中的主导瓶颈。

7.4 在线服务

方法。我们评估不同每秒智能体到达率 (APS) 下的系统延迟特征。智能体按指定速率服从泊松过程到达；每个智能体到达后，从第 0 轮开始 replay 到最后一轮。实验中，SLO 设置为 $TTFT \leq 4$ 秒且 $TPOT \leq 50ms$ 。在 TPOT 和 TTST 图中，超过 SLO 阈值的数据点被省略。当出现以下任一情况时，实验终止：(1)

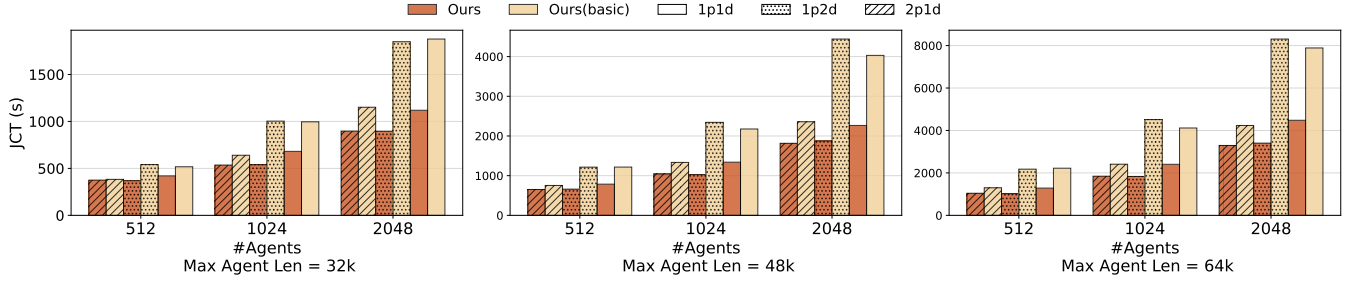


图 8: Prefill-decode 比例对离线推理性能的影响 (DS 27B)。

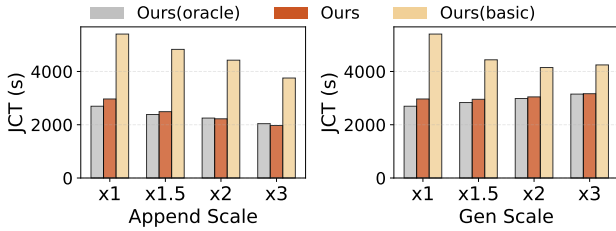


图 9: 左: 不同追加长度 (DS 660B, 64K 上下文, 1024 个智能体)。右: 不同生成长度 (DS 660B, 64K, 1024 个智能体)。

TTFT 超过 4 秒; 或 (2) 系统达到稳态, 即 150 秒滑动窗口内 TTFT 变化相较 30 分钟前保持在 5% 以下。

如 Figure 10 所示, DualPath 相比 Basic 可支持更高 APS 容量 (DS 27B 为 1.67 $\times$, DS 660B 为 2.25 $\times$)。DualPath 的 TTST 与 Basic 相当, 而 TPOT 表明, 相比 Basic, DualPath 不会引入额外 decode 开销。SGL(MC) 表现出异常低的 TTST, 这可能是实现问题导致的: 前两个 token 几乎同时到达客户端。对于 DS 27B, 所有指标都呈现出与 DS 660B 类似的趋势。然而, Basic 和 DualPath 的 TPOT 都显著高于 Oracle, 说明在小模型场景下基础 P-D 传输开销相当可观。我们将其留作未来工作。

两个模型的平均 JCT 如 Figure 11 所示。关于 working set 影响的详细分析见 §8。如 Figure 12 (左) 所示, DualPath 在不同 APS 下保持稳定的 TTFT 组成, 而 Basic 由于存储带宽不足, 其排队时间急剧增长。

7.5 消融研究

我们进行消融研究, 以量化 DualPath 中每个技术组件的贡献。实验在离线推理设置下进行, MAL 为 64K, 智能体 batch size 为 1024 和 2048。Basic 与 Ours 之间的差异被分为三项技术: layerwise prefill、dual-path loading 和 scheduling algorithm。我们逐步加入这些技术, 以展示各自贡献。如 Figure 12 所示, 相比 Basic, 加入 layerwise prefill 平均降低 JCT 17.21%, 缓解 PE HBM 瓶颈并隐藏传输开销。在 layerwise prefill 之上加入 Dual-path loading 带来主要性能收益, 相比 Basic 平均降低 JCT 38.19%, 因为它允许请求从 PE 或 DE 任一侧读取 KV-Cache, 充分利用分布式存储带宽。最后, 在 dual-path loading 之上采用我们的调度算法来决定 KV-Cache 加载路径, 可获得最佳性能, 相比 Basic 降低 JCT 45.62%, 证明跨存储 NIC 的负载均衡调度是有效的。

负载均衡。 DualPath 的调度算法改善了存储 NIC 和 attention 层执行时间两方面的负载均衡。对于存储 NIC, 相比 round robin 调度, 我们的调度算法将负载均衡指标从 1.53 改善到 1.18 (Figure 13)。对于 attention 层, 在任务最初 5% 期间, DualPath 将 Max/Avg 比率维持在低至 1.06, 从而减少 GPU idle bubble (Figure 14)。存储 NIC 指标是在一个小时窗口内, 三台机器上所有存储 NIC 的最大流量与平均流量之比, 其中 1.0 表示完美均衡。Attention 层指标针对每个 forward, 在 expert parallel group 内所有 GPU 之间计算。随着任务推进, 由于系统负载不

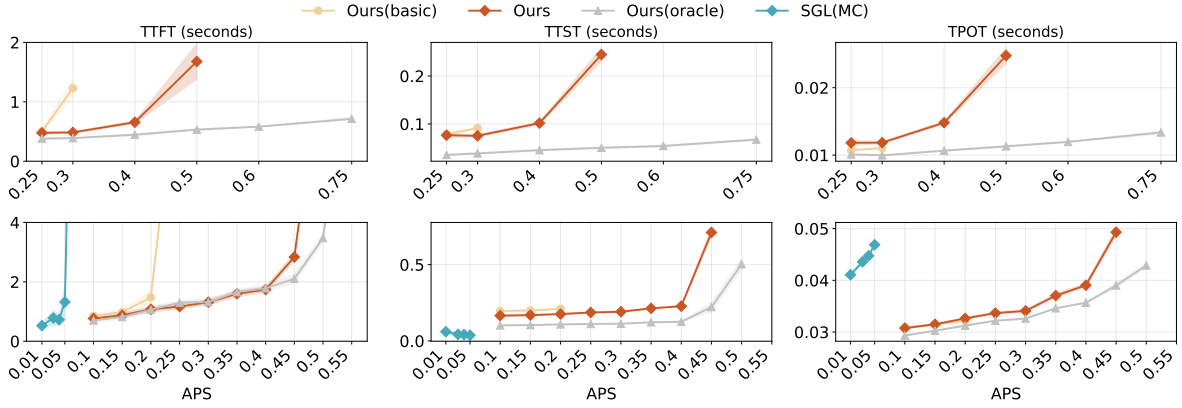


图 10: TTFT、TTST 和 TPOT 随智能体到达率 (APS) 的变化。阴影表示实验结束前最后 150s 内的波动。上: DS 27B; 下: DS 660B。

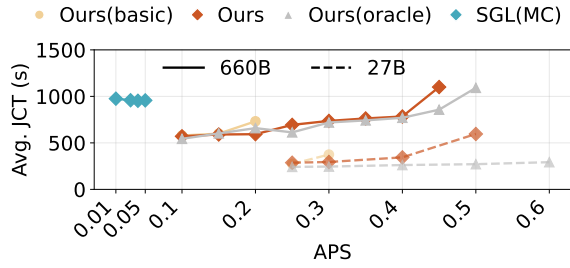


图 11: 在线服务中, 所有轨迹平均完成时间随到达率的变化。

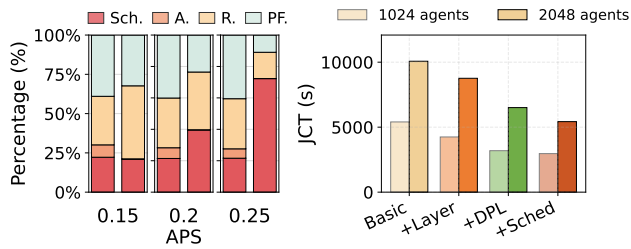


图 12: 左 (§7.4): 在线服务的 TTFT 细分 (DS 660B), 横跨不同 APS; Sch. 表示 scheduling, A. 表示 allocating, R. 表示 reading KV-cache, PF. 表示 prefill. 每对柱子中, 第一个为 DualPath, 第二个为 Basic. 右 (§7.5): 离线推理消融结果 (DS 660B, 64K 上下文长度)。Layer、DPL、Sched 分别表示 layerwise prefill、Dual-Path Loading 和 scheduling。

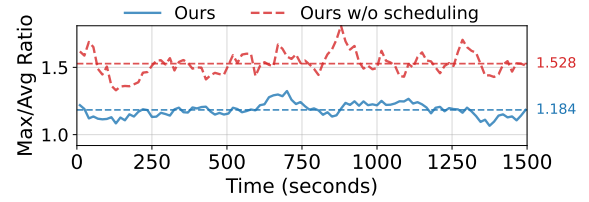


图 13: 存储 NIC 流量负载均衡。

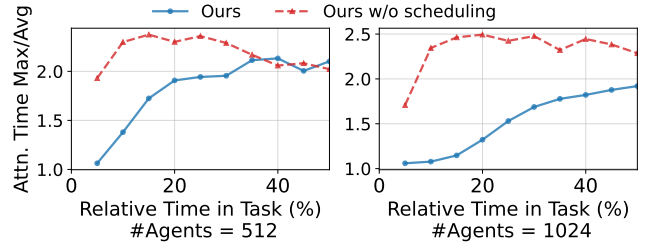


图 14: Attention 执行时间的负载均衡。

表 3: 大规模实验结果。

	Setting	JCT	TTFT	TTST	TPOT
Offline	2P4D, 2K agents	3,167s	—	—	—
	48P96D, 48K agents	3,201s	—	—	—
Online	2P4D, 0.4 APS	—	1.739s	0.228s	0.039s
	44P88D, 8.8 APS	—	1.847s	0.194s	0.036s

足, 两个比率都会失去意义。因此, 我们没有展示工作负载的尾部阶段。

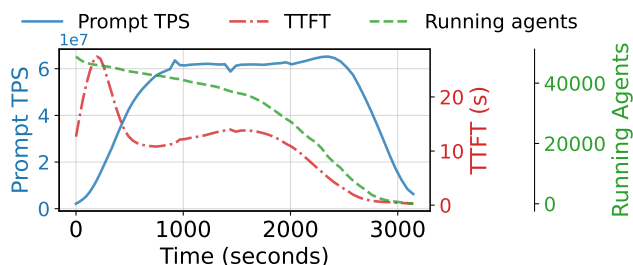


图 15: 48P96D 离线推理指标。1e7 是 Prompt TPS 的缩放因子。

7.6 大规模可扩展性

我们使用最多 1,152 块 GPU 进行离线和在线实验，以展示大规模可扩展性 (Table 3)。对于离线推理，从 2P4D (2K 个智能体) 扩展到 48P96D (48K 个智能体) 时，在相近 JCT 下获得近线性加速 (3,167s vs. 3,201s)。对于在线服务，44P88D 配置在保持相近延迟的同时实现 22× 吞吐量 (8.8 vs. 0.4 APS)。所有实验中，scheduler CPU 使用量始终低于 10 个 core，说明它不是瓶颈。离线推理过程中的若干详细指标见 Figure 15。

由于缺少充分调优的并行设置和 P/D 比例（这需要大量实验预算），大规模实验并未相比等价成本下的多个小规模单元展示额外 JCT 或服务容量收益。然而，大规模设置仍然重要，原因如下。第一，它减少碎片化，并为调优并行方式和 P/D 比例提供更大灵活性。第二，在不可预测的突发在线请求下，大规模部署提供更多调度机会来缓解排队延迟。这些观察指向若干未来工作方向 (§8.1)。

8 讨论

8.1 潜在未来工作

离线推理的工作负载高度动态。例如，在我们的智能体 RL 任务中，工作负载很大程度上取决于研究者的算法设计，并且 prefill 阶段通常在执行前半段承受的压力显著高于后半段。同时，对这些试探性实验进行 profiling 成本很高，因为有些实验只会运行有限次数。

因此，需要更自适应且更灵活的并行方式和 P/D 比例配置方法，例如模拟器或在线调整机制。其次，调度算法仍有改进空间，因为我们期望在大规模部署中获得更低的 TTFT 分位数。

8.2 Working Set 分析

如 Figure 11 所示，给定到达率 λ (即每秒新轨迹数) 和平均 JCT $\bar{t}$, KV-Cache 的 working set 可近似为 $\lambda \bar{t} \times total_len_{avg}/2$ 。在 DS 660B 服务设置中，DualPath 的该值从 APS 0.1 时的 69 GB 到 APS 0.45 时的 681 GB 不等。

在真实设置中，working set 会更大，因为我们的评估假设 inter-arrival time 为零且 tool call latency 为零。如果由于这些间隔导致 JCT 增加 r 倍，则系统 APS 容量也会增加 r 倍（这些间隔不会对 LLM 推理造成压力），使 working set 扩大 r^2 倍。这会超过可用内存并降低分布式内存池命中率。这类实验需要 r 倍更多机器小时和 r^2 倍更多存储（成本按 r^3 缩放），在有限资源下我们无法承担。

9 相关工作

分布式内存缓存池。 Mooncake [38] 为 KV-Cache 构建了分布式 DRAM 池。TokenLake [46] 引入了统一的 segment-level prefix cache 池。与它们相比，DualPath 直接面向存储后端，在所有 SNIC 之间均衡流量，并在不损害性能的情况下大幅减少 DRAM 使用。DualPath 也可以与中间 DRAM cache 结合，但性能收益较小。

KV-Cache I/O 优化。 在解耦式 LLM serving 架构中，从其他缓存层级高效加载大规模 KV-Cache 是一个根本瓶颈。已有工作主要从单条数据路径角度处理这一问题。Strata [49] 通过协同设计 GPU 辅助 I/O 与 cache-aware scheduling，解决层次化存储中的 I/O 瓶颈。其他工作如 KVPR [23] 和 TailorKV [53]，通过重计算重叠和层粒度混合量化缓解该路径上的带宽限制（例如 PCIe）。

LLM 推理系统。近年来出现了许多推理加速技术, 例如 paged attention [26]、chunked prefill 和 hybrid batching [2, 21]。Prefill-decode 解耦推理 [37, 58] 将 prefill 和 decode 阶段分离到不同 GPU 上, 减少二者之间的性能干扰, 并允许每个阶段采用不同并行策略和硬件配置, 从而释放大量优化机会。它已经基本成为推理服务的事实标准。

Attention 机制。Attention 机制允许 token 与序列中的先前 token 交互。它有许多变体, 例如 Multi-Head Attention (MHA) [44]、Multi-Query Attention (MQA) [41]、Grouped-Query Attention (GQA) [3], 以及 Multi-head Latent Attention (MLA) [12]。对于这些 attention 机制 (记为 *dense attention*), 单个 token 的计算量与 KV-Cache 大小之比是常数, 因为二者都随序列长度线性增长。

10 结论

本文提出 DualPath, 一个面向智能体 LLM 推理的框架, 通过双路径 KV-Cache 加载解决 PD 解耦架构下 KV-Cache 读取不均衡问题。通过结合工作负载感知调度重新分配存储网络负载, DualPath 在离线推理中最高实现 1.87 $\times$ 吞吐量提升。在在线服务中, 它也使每秒完成的智能体运行数量平均提升 1.96 $\times$ 。

REFERENCES

- [1] 2026. UnifiedBus. <https://www.unifiedbus.com/en>. (2026).
- [2] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming Throughput-Latency Trade-off in LLM Inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 117–134.
- [3] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebron, and Sumit Sanghai. 2023. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 4895–4901.
- [4] Anthropic. 2026. Introducing Claude Opus 4.6. <https://www.anthropic.com/news/claude-opus-4-6>. (2026).
- [5] InfiniBand Trade Association. 2007. InfiniBand Architecture Specification Volume 1, Release 1.2.1. (2007).
- [6] Brian E. Carpenter and Kathleen M. Nichols. 2002. Differentiated services in the Internet. *Proc. IEEE* 90 (2002), 1479–1494. <https://api.semanticscholar.org/CorpusID:1723205>
- [7] Qiaoling Chen, Zhisheng Ye, Tian Tang, Peng Sun, Boyu Tian, Guoteng Wang, Shenggui Li, Yonggang Wen, Zhenhua Han, and Tianwei Zhang. 2026. CONCUR: High-Throughput Agentic Batch Inference of LLM via Congestion-Based Concurrency Control. (2026). arXiv:cs.DC/2601.22705 <https://arxiv.org/abs/2601.22705>
- [8] Sadia Sultana Chowh, Riasad Alvi, Subhey Sadi Rahman, Md Abdur Rahman, Mohaimenul Azam Khan Raiaan, Md Rafiqul Islam, Mukhtar Hussain, and Sami Azam. 2026. From language to action: a review of large language models as autonomous agents and tool users. *Artificial Intelligence Review* (2026).
- [9] Ultra Ethernet Consortium. 2026. Ultra Ethernet Specification v1.0.2. (2026).
- [10] Tri Dao. 2024. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. In *International Conference on Learning Representations*. 35549–35562.
- [11] Google DeepMind. 2026. Gemini 3 Pro. <https://deepmind.google/models/gemini/pro/>. (2026).
- [12] DeepSeek-AI. 2024. DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model. (2024). arXiv:2405.04434
- [13] DeepSeek-AI. 2025. 3FS. <https://github.com/deepseek-ai/3FS>. (2025).
- [14] DeepSeek-AI. 2025. DeepGEMM. <https://github.com/deepseek-ai/DeepGEMM>. (2025).
- [15] DeepSeek-AI. 2025. DeepSeek-V3 Technical Report. (2025). arXiv:cs.CL/2412.19437 <https://arxiv.org/abs/2412.19437>
- [16] DeepSeek-AI. 2025. DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models. (2025). arXiv:2512.02556
- [17] Kuntai Du, Bowen Wang, Chen Zhang, Yiming Cheng, Qing Lan, Hejian Sang, Yihua Cheng, Jiayi Yao, Xiaoxuan Liu, Yifan Qiao, Ion Stoica, and Junchen Jiang. 2025. PrefillOnly: An Inference Engine for Prefill-only Workloads in Large Language Model Applications. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles (SOSP '25)*. 399–414.
- [18] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. 2024. Cost-Efficient Large Language Model Serving for Multi-turn Conversations with CachedAttention. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*. 111–126.
- [19] Shiwei Gao, Youmin Chen, and Jiwei Shu. 2025. Fast State Restoration in LLM Serving with HCache. In *Proceedings of the 20th European Conference on Computer Systems*. 128–143.
- [20] Chuanxiong Guo, Haitao Wu, Zhong Deng, Gaurav Soni, Jianxi Ye, Jitu Padhye, and Marina Lipshteyn. 2016. RDMA over Commodity Ethernet at Scale. In *Proceedings of the 2016 ACM SIGCOMM Conference (SIGCOMM '16)*. Association for Computing Machinery, New York, NY, USA, 202–215. <https://doi.org/10.1145/2934872.2934908>

- [21] Connor Holmes, Masahiro Tanaka, Michael Wyatt, Ammar Ahmad Awan, Jeff Rasley, Samyam Rajbhandari, Reza Yazdani Aminabadi, Heyang Qin, Arash Bakhtiari, Lev Kurilenko, and Yuxiong He. 2024. DeepSpeed-FastGen: High-throughput Text Generation for LLMs via MII and DeepSpeed-Inference. (2024). arXiv:2401.08671
- [22] Yifan Hu, Shi Qiu, Jianqin Yan, Hao Chen, Xintao Wang, Tang Lu, Guangtao Xue, and Yiming Zhang. 2025. TARDIS: A GPU-Centric KV Cache Service for Efficient LLM Inference. In *Proceedings of the 16th ACM SIGOPS Asia-Pacific Workshop on Systems (APSys '25)*. Association for Computing Machinery, New York, NY, USA, 46–53. <https://doi.org/10.1145/3725783.3764393>
- [23] Chaoyi Jiang, Lei Gao, Hossein Entezari Zarch, and Murali Annavaram. 2025. KVPR: Efficient LLM Inference with I/O-Aware KV Cache Partial Recomputation. In *Findings of the Association for Computational Linguistics: ACL 2025*. 19474–19488.
- [24] Juyong Jiang, Fan Wang, Jiasi Shen, Sungju Kim, and Sunghun Kim. 2024. A survey on large language models for code generation. *ACM Transactions on Software Engineering and Methodology* (2024).
- [25] Anuj Kalia, Michael Kaminsky, and David G Andersen. 2016. Design guidelines for high performance RDMA systems. In *2016 USENIX annual technical conference (USENIX ATC 16)*. 437–450.
- [26] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 611–626.
- [27] Jiashi Li and Shengyu Liu. 2025. FlashMLA: Efficient Multi-head Latent Attention Kernels. <https://github.com/deepseek-ai/FlashMLA>. (2025).
- [28] Yuanchun Li, Hao Wen, Weijun Wang, Xiangyu Li, Yizhen Yuan, Guohong Liu, Jiacheng Liu, Wenxing Xu, Xiang Wang, Yi Sun, Rui Kong, Yile Wang, Hanfei Geng, Jian Luan, Xuefeng Jin, Zilong Ye, Guanqing Xiong, Fan Zhang, Xiang Li, Mengwei Xu, Zhijun Li, Peng Li, Yang Liu, Ya-Qin Zhang, and Yunxin Liu. 2024. Personal LLM Agents: Insights and Survey about the Capability, Efficiency and Security. (2024). arXiv:cs.HC/2401.05459 <https://arxiv.org/abs/2401.05459>
- [29] Weizhe Lin, Hui-Ling Zhen, Shuai Yang, Xian Wang, Renxi Liu, Hanting Chen, Wangze Zhang, Chuansai Zhou, Yiming Li, Chen Chen, Xing Li, Zhiyuan Yang, Xiaosong Li, Xianzhi Yu, Zhenhua Dong, Mingxuan Yuan, and Yunhe Wang. 2025. Towards Efficient Agents: A Co-Design of Inference Architecture and System. (2025). arXiv:cs.CL/2512.18337 <https://arxiv.org/abs/2512.18337>
- [30] Yuhua Liu, Yihua Cheng, Jiayi Yao, Yuwei An, Xiaokun Chen, Shaoting Feng, Yuyang Huang, Samuel Shen, Rui Zhang, Kuntai Du, and Junchen Jiang. 2025. LMCACHE: An Efficient KV Cache Layer for Enterprise-Scale LLM Inference. (2025). arXiv:2510.09665
- [31] Mahmoud Mohammadi, Yipeng Li, Jane Lo, and Wendy Yip. 2025. Evaluation and benchmarking of llm agents: A survey. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*. 6129–6139.
- [32] NVIDIA. 2023. SuperPOD: Next Generation Scalable Infrastructure for AI Leadership. <http://docs.nvidia.com/https://docs.nvidia.com/dgx-superpod-reference-architecture-dgx-h100.pdf>. (2023).
- [33] NVIDIA. 2026. Developing a Linux Kernel Module using GPUDirect RDMA. <https://docs.nvidia.com/cuda/gpudirect-rdma/index.html>. (2026).
- [34] NVIDIA. 2026. GPUDirect Storage Overview Guide. <https://docs.nvidia.com/gpudirect-storage/overview-guide/index.html>. (2026).
- [35] OpenAI. 2025. gpt-oss-120b & gpt-oss-20b Model Card. (2025). arXiv:2508.10925
- [36] OpenAI. 2025. Introducing GPT-5.2. <https://openai.com/index/introducing-gpt-5-2/>. (2025).
- [37] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2025. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *Proceedings of the 51st Annual International Symposium on Computer Architecture*. 118–132.
- [38] Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2025. Mooncake: Trading More Storage for Less Computation — A KVCache-centric Architecture for Serving LLM Chatbot. In *Proceedings of the 23rd USENIX Conference on File and Storage Technologies*. 155–170.
- [39] Andre Richter, Christian Herber, Thomas Wild, and Andreas Herkersdorf. 2016. Resolving Performance Interference in SR-IOV Setups with PCIe Quality-of-Service Extensions. In *2016 Euromicro Conference on Digital System Design (DSD)*. 454–462. <https://doi.org/10.1109/DSD.2016.41>
- [40] SGLang. 2026. SGLang HiCache. https://docs.sglang.io/advanced_features/hicache.html. (2026).
- [41] Noam Shazeer. 2019. Fast Transformer Decoding: One Write-Head is All You Need. (2019). arXiv:1911.02150
- [42] Qwen Team. 2025. Qwen2.5 Technical Report. (2025). arXiv:2412.15115
- [43] Qwen Team. 2025. Qwen3 Technical Report. (2025). arXiv:cs.CL/2505.09388 <https://arxiv.org/abs/2505.09388>
- [44] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is All You Need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*. 6000–6010.
- [45] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, et al. 2024. A survey on large language model based autonomous agents. *Frontiers of Computer Science* 18, 6 (2024), 186345.
- [46] Bingyang Wu, Zili Zhang, Yinmin Zhong, Guanzhe Huang, Yibo Zhu, Xuanchu Liu, and Xin Jin. 2025. TokenLake: A Unified

- Segment-level Prefix Cache Pool for Fine-grained Elastic Long-Context LLM Serving. (2025). arXiv:cs.DC/2508.17219 <https://arxiv.org/abs/2508.17219>
- [47] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W White, Doug Burger, and Chi Wang. 2023. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. (2023). arXiv:cs.AI/2308.08155 <https://arxiv.org/abs/2308.08155>
- [48] Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, et al. 2025. The rise and potential of large language model based agents: A survey. *Science China Information Sciences* 68, 2 (2025), 121101.
- [49] Zhiqiang Xie, Ziyi Xu, Mark Zhao, Yuwei An, Vikram Sharma Mailthody, Scott Mahlke, Michael Garland, and Christos Kozyrakis. 2025. Strata: Hierarchical Context Caching for Long Context Language Model Serving. (2025). arXiv:2508.18572
- [50] Yi Xiong, Hao Wu, Changxu Shao, Ziqing Wang, Rui Zhang, Yuhong Guo, Junping Zhao, Ke Zhang, and Zhenxuan Pan. 2024. LayerKV: Optimizing Large Language Model Serving with Layer-wise KV Cache Management. (2024). arXiv:2410.00428
- [51] Jianqin Yan, Shi Qiu, Yina Lv, Yifan Hu, Hao Chen, Zhirong Shen, Xin Yao, Renhai Chen, Jiwu Shu, Gong Zhang, and Yiming Zhang. 2025. Phoenix: A Refactored I/O Stack for GPU Direct Storage without Phony Buffers. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '25)*. 1267–1283.
- [52] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. Swebagent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems* 37 (2024), 50528–50652.
- [53] Dingyu Yao, Bowen Shen, Zheng Lin, Wei Liu, Jian Luan, Bin Wang, and Weiping Wang. 2025. TailorKV: A Hybrid Framework for Long-Context Inference via Tailored KV Cache Optimization. In *Findings of the Association for Computational Linguistics: ACL 2025*. 20340–20359.
- [54] Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, and Luis Ceze. 2025. FlashInfer: Efficient and Customizable Attention Engine for LLM Inference Serving. (2025). arXiv:2501.01005
- [55] Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Huazuo Gao, Jiashi Li, Liyue Zhang, Panpan Huang, Shangyan Zhou, Shirong Ma, et al. 2025. Insights into deepseek-v3: Scaling challenges and reflections on hardware for ai architectures. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture*. 1731–1745.
- [56] Chenggang Zhao, Shangyan Zhou, Liyue Zhang, Chengqi Deng, Zhean Xu, Yuxuan Liu, Kuai Yu, Jiashi Li, and Liang Zhao. 2025. DeepEP: an efficient expert-parallel communication library. <https://github.com/deepseek-ai/DeepEP>. (2025).
- [57] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2024. SGLang: Efficient Execution of Structured Language Model Programs. In *Proceedings of the 38th International Conference on Neural Information Processing Systems*. Article 2000, 27 pages.
- [58] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 193–210.
- [59] Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, et al. 2023. Webarena: A realistic web environment for building autonomous agents. *arXiv preprint arXiv:2307.13854* (2023).

A 附录

A.1 流量隔离配置细节

InfiniBand. InfiniBand QoS 机制采用两个仲裁器：高优先级仲裁器和低优先级仲裁器。流量先在高优先级仲裁器中使用 Weighted Round Robin (WRR) 调度，然后根据 `qos_high_limit` 被引导到低优先级仲裁器；将其设置为 255 会完全禁用低优先级仲裁器。详细调度算法见 [5]。我们的配置如下：

- `qos_max_vls 4`
- `qos_high_limit 240`
- `qos_vlarb_high 0:192,1:192,2:0,3:192`
- `qos_vlarb_low 0:192,1:192,2:64,3:192`

RoCE. RoCE 通过基于 DSCP 的流量分类和硬件 Traffic Class (TC) 执行 QoS。Packet 首先从 DSCP 值映射到 TC，每个 TC 都由 NIC 和 switch 上的专用硬件队列承载（通常最多 8 个）。为匹配 InfiniBand 中的四 VL 配置，我们配置 4 个无损 RDMA TC，并启用 Priority Flow Control (PFC)。通过在 NIC 和 switch 上为这些 TC 分配成比例的调度权重，实现带宽隔离：为模型推理流量预留大部分带宽，同时为 KV-cache 流量分配小部分带宽以防饥饿。

A.2 27B 模型规格

从整体模型规模看, hidden dimension 为 2560, dense layer 的 intermediate size 为 12288, hidden layer 数量为 30, attention head 数量为 32, routed expert 数量为 72, MoE intermediate size 为 1536, 每个 token 激活 expert 数量为 6, shared expert 数量为 2, 初始 dense layer 数量为 1。关于 index attention 机制, attention head 数量为 32, head dimension 为 64, sparse attention 的 topk token 数为 1024。Indexer 和 main attention 的 Q 矩阵 LoRA 压缩被移除。

A.3 智能体任务结构

为解释数据集特征,我们简要描述智能体任务结构;不过该背景与我们的系统设计正交。每个智能体运行在 sandbox 环境中,环境包含一个带有已知 bug 和相关错误信息的代码仓库。智能体通过 prompt 被要求诊断并修复 bug。模型具备工具使用能力,可通过生成结构化输出来在 sandbox 中调用 bash 命令。智能体和环境进行多轮交互,每一轮由一个 prompt (先前上下文与新信息拼接而成,其中大部分是工具输出)和模型通过 decode 生成的后续工具调用组成。

每条轨迹是一系列轮次;第 i 轮包含追加 token A_i 和生成 token 数 g_i 。我们用 G_i 表示第 i 轮生成的 token, 这些 token 未在数据集中直接呈现。我们定义 $Context_{i+1}$ 为 $A_1, G_1, A_2, G_2, \dots, A_i, G_i$ 的拼接列表。在 replay 的第 $i+1$ 轮中,智能体将 prompt 拼接为 $Context_{i+1} + A_{i+1}$, 随后设置合适采样参数,确保它恰好生成 g_{i+1} 个 token, 即 G_{i+1} 。为生成额外智能体轨迹,我们采样一条已有轨迹,并在其前面添加一个合成轮次,其中 A_1 为随机 token, 且 $g_1 = 1$ 。

A.4 实验配置

配置参数。对于 DeepSeek 模型, DualPath 每个节点分配 80GB DRAM, 而 SGL(MC) 每个节点总共使用 1.5TB DRAM。对于 Qwen 32B, 由于 KV-Cache 更大, DualPath 分配 320GB。所有设置都禁用 speculative decoding。所有配置都使用 3FS 作为存储后端。短读取队列阈值 α (见 §6) 设置为 3 秒内可读取的 token 数, 未完成 token 上限 β 设置为单块 GPU 在 5 秒内可处理的 token 数。这些值都提前 profiling 得到。所有 DualPath 和 Oracle 基线的 Compute Quota Threshold 均设置为 300ms。

KV-Cache 命中长度计算。对于除 SGL(MC) 以外的所有系统,我们限制 KV-Cache 命中只发生在同一条轨迹内,且由于无需 eviction,命中长度在 client 侧计算。对于 SGL(MC),命中长度基于 HiCache 和 Mooncake Store cache 状态在内部计算。

A.5 KV-Cache Block 布局

逐层 prefill 将 KV-Cache block size 降为原来的 $1/layer$, 并使 block 数量增至 $layer \times$, 这对传输和存储性能提出挑战。为克服这一点,我们设计了两种不同 block 类型: *Layer Block* 和 *Full Block*。*Layer Block* 是形状为 $[1, tokens, bytes]$ 的字节张量,用于存储若干 token 的单层 KV-Cache。token 数称为 *block_size.tokens*, *bytes* 表示每层每 token 所需的 cache 字节数。同时, *Full Block* 的形状为 $[layer, tokens, bytes]$ 。该设计使我们可以整个推理过程中避免手动 KV-Cache 内存布局转换,只需拼接 n 个 *Layer Block* 即可得到一个 *Full Block*。KV-Cache 使用 trie 结构存储在分布式存储中,其中每个树节点对应一个 *Full Block*。